

制作物：コード生成ツール

- ソースコードを解析して、シリアライズ処理やエディタ表示処理など、定型的なコードを生成するツール
- 自動生成することで、作業時間の短縮、手作業によるミスの削減をするために開発をした

```
#include "Transform.generated.h"
class [[MT_COMPONENT()]] Transform :
{
public:
    MT_GENERATED_BODY()

    friend TransformCP;
    using IComponent::IComponent;

    [[MT_PROPERTY()]]
    EntityId parent;
    [[MT_PROPERTY()]]
    Vector3 position;
    [[MT_PROPERTY()]]
    Vector3 scale;
    [[MT_PROPERTY()]]
    Quaternion rotate;
```

生成されたファイル

クラス登録

処理の展開部分

変数登録



コード生成

Undo/Redo
シリアライズ
エディタ表示、編集

開発メンバー：一人(自作)

開発期間：約8か月(ゲームや他ツールと並行して開発)

所要時間：約80時間

使用言語：C#

使用ライブラリ：

ClangSharp(ソースコード解析)

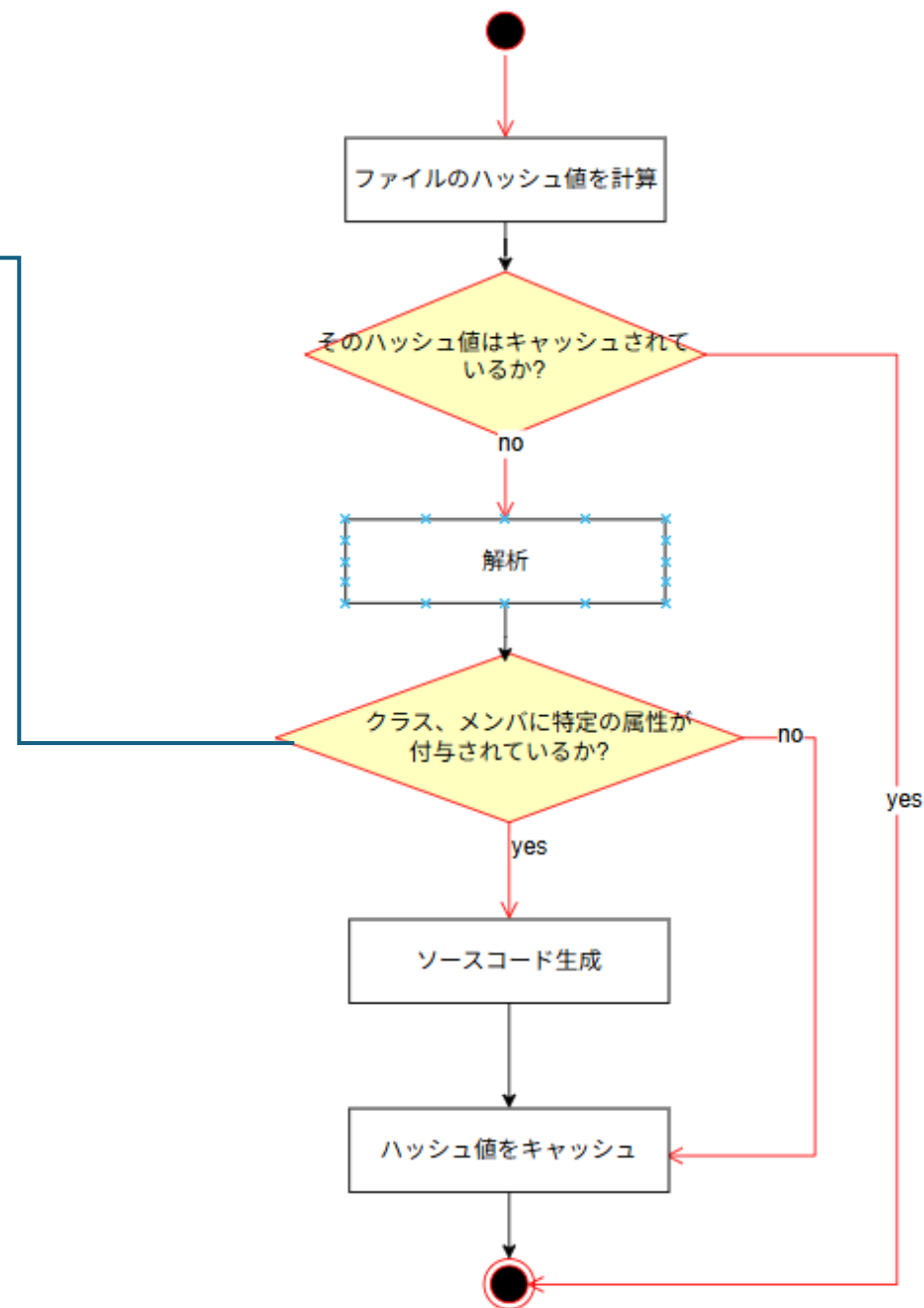
Scriban(コード生成)

YamlDotNet、Newtonsoft.Json(YAML,JSON操作)

流れ

```
#define MT_STRINGIFY(...) #__VA_ARGS__  
#define MT_PROPERTY(...) clang::annotate("MT_PROPERTY," MT_STRINGIFY(__VA_ARGS__))  
#define MT_FUNCTION(...) clang::annotate("MT_FUNCTION," MT_STRINGIFY(__VA_ARGS__))  
#define MT_COMPONENT(...) clang::annotate("MT_COMPONENT," MT_STRINGIFY(__VA_ARGS__))  
#define MT_GENERATED_BODY()  
  
class [[MT_COMPONENT()]] Transform :  
-> [[MT_PROPERTY()]]  
-> EntityId parent;  
-> [[MT_PROPERTY()]]  
-> Vector3 position;  
-> [[MT_PROPERTY()]]  
-> Vector3 scale;  
-> [[MT_PROPERTY()]]  
-> Quaternion rotate;
```

clang::annotateで属性を付与



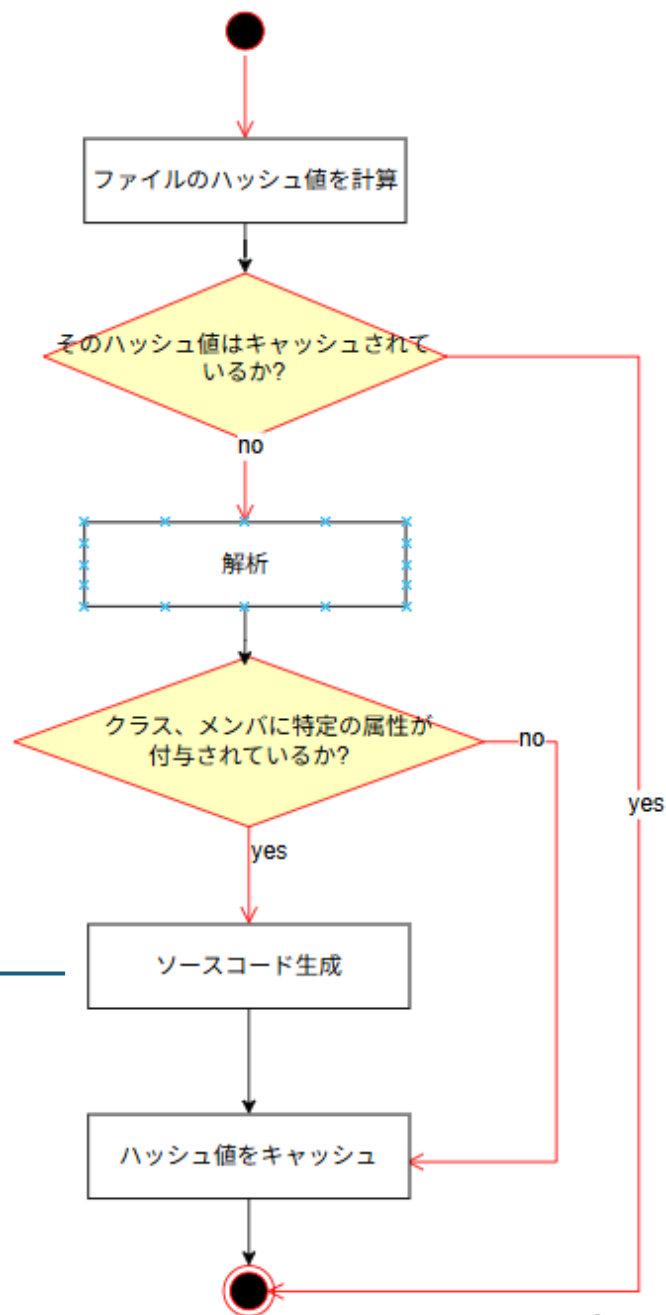
流れ

クラス情報をScribanテンプレートエンジンに渡して、
generated.hを生成する

```
// =====  
// {{ class_name }}の状態を保存するState構造体の定義、Undo/Redoに使う  
// =====  
struct {{ class_name }}State  
{  
    {{- for member in properties}}  
    | {{ get_fully_qualified_type(member) }} {{ member.name }};  
    {{- end }}  
};
```



```
// =====  
// Transformの状態を保存するState構造体の定義、  
// =====  
struct TransformState  
{  
    → mtgb: EntityId parent;  
    → mtgb: Vector3 position;  
    → mtgb: Vector3 scale;  
    → mtgb: Quaternion rotate;  
};
```

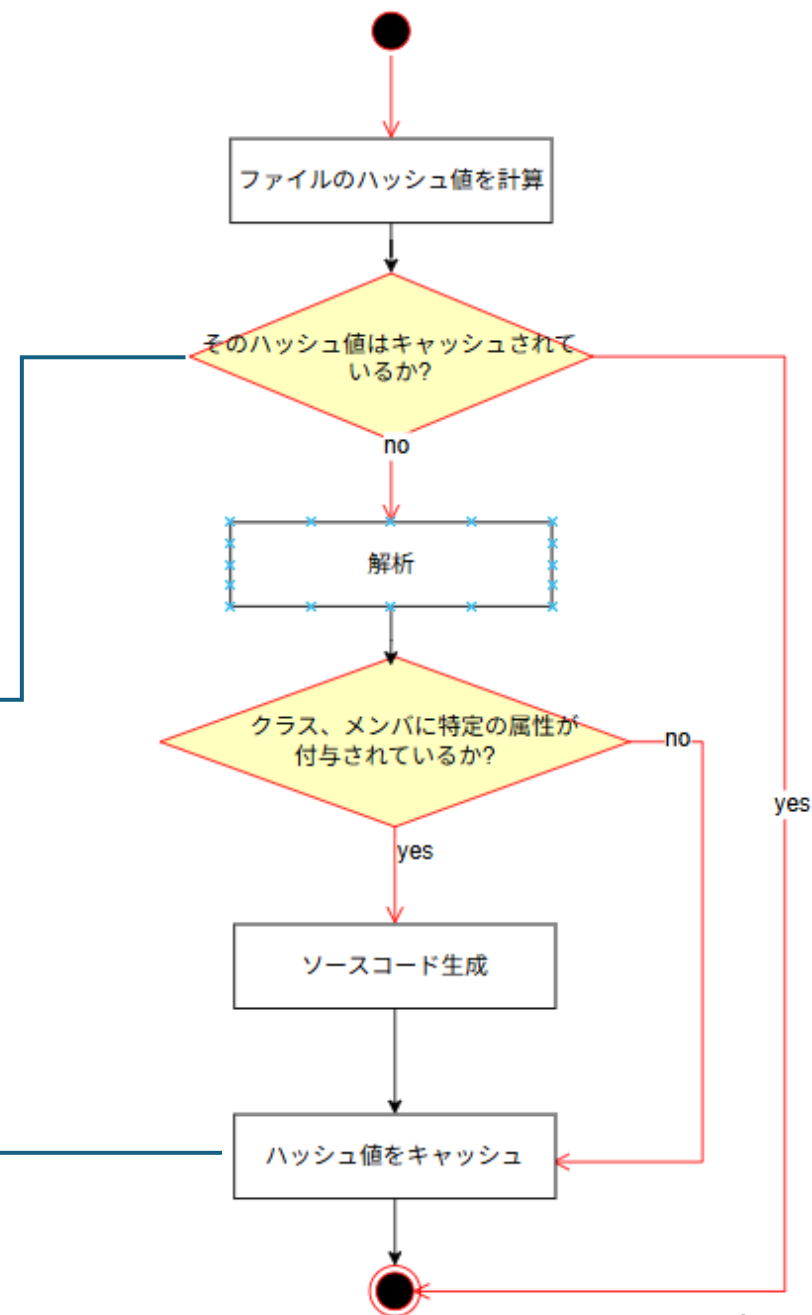


流れ

ファイルのハッシュ値を記録し、変更されたファイルのみ解析

```
var currentHash = XxHashComputer.ComputeHash(filePath);  
if (memoryCache.TryGetValue(filePath, out CacheEntry? entry) == false)  
{  
    // キャッシュにない  
    return true;  
}  
if (entry == null)  
{  
    return true;  
}  
if (currentHash != entry.FileHash)  
{  
    // ハッシュが異なる  
    return true;  
}
```

```
string hash = XxHashComputer.ComputeHash(filePath);  
lock (lockObj)  
{  
    // ハッシュとその時の時間を保存  
    memoryCache[filePath] = new CacheEntry  
    {  
        FileHash = hash,  
        LastAnalyzed = DateTime.UtcNow  
    };  
}
```



操作方法

コマンド

```
使用法:  
ClangSourceGenerator [options]
```

オプション:

--f, --force	キャッシュを消去して、全ファイルを再解析
--config	解析の設定ファイルのディレクトリ デフォルトでプロジェクトのルート
--projDir, --projectDirectory	プロジェクトのディレクトリ デフォルトでプロジェクトのルート
-, -h, --help	ヘルプ表示

操作方法

ビルド前に実行する場合

msbuild

- .vcxprojにビルド前タスクとして設定

```
<!--自動実行: EchoProjectDirPath→GenerateCodeの順序で実行-->
<Target Name="GenerateCodeTarget" BeforeTargets="ClCompile">
  <!--コード生成ツール実行-->
  <Exec Command="&quot;$(ProjectDir)Tools¥ClangTest.exe&quot; &quot;$(ProjectDir)&quot;"/>
</Target>
<!--手動実行: 全ファイルを再生成する-->
<Target Name="ForceRegenerateCode">
  <Exec Command="&quot;$(ProjectDir)Tools¥ClangTest.exe&quot; &quot;$(ProjectDir)&quot; --force"/>
</Target>
```

```
msbuild Game.sln -t:GenerateCodeTarget
ForceRegenerateCode
```

CMake

- add_dependenciesでビルド前に実行するよう設定

```
add_custom_target(generate_source_code
  COMMAND "$<SHELL_PATH:$[CMAKE_CURRENT_SOURCE_DIR]/Tool
  USES_TERMINAL)
add_dependencies(${PROJECT_NAME} generate_source_code)
add_custom_target(force_generate_source_code
```

```
cmake --build out/build/default-debug --target generate_source_code
force_generate_source_code
```

操作方法

設定ファイル

- .clang-src-gen.yamlを配置する必要がある

```
ExcludeDirectories:  
  - "EffekseeLib"  
  - "vcpkg_installed"  
  - "ImGui"  
  - "packages"  
MaxDegreeOfParallelism: 4  
CodeGenerationRules:  
  - ClassMetadataType: "MT_COMPONENT"  
  - MemberMetadataType: "MT_PROPERTY"  
  - OutputTemplate: "GenerateComponentHeader.sbn"
```


オプション

ExcludeDirectories	解析対象から除外するディレクトリ
MaxDegreeOfParallelism	並列処理の最大数 ※ 0以下の場合、使用可能な論理プロセッサ数が割り当てられるので注意
OutputDirectory	生成ファイルが配置されるフォルダ

オプション

CodeGenerationRules	生成するソースコードの指定、生成対象とするクラス、メンバの条件
- ClassMetadataType	クラスの属性
- MemberMetadataType	メンバの属性
- OutputTemplate	生成するソースコードのテンプレート(scriban)
- OutputFileName	生成されるファイル名 “{}.generated.h” という文字列にすると、“{}”が解析したクラス名に置換される

Scriban テンプレート

- 解析した情報はテンプレート内で使用することができる。
- 下の画像のように、Scribanの構文を書いてひな形を作成する。

<https://scriban.github.io/>
公式のドキュメント

```
// =====  
// {{ class_name }}の状態を保存するState構造体の定義、Undo/Redoに使う  
// =====  
struct {{ class_name }}State  
{  
    {{- for member in properties}}  
    |    {{ get_fully_qualified_type(member) }} {{ member.name }};  
    {{- end }}  
};
```

テンプレート



```
// =====  
// Transformの状態を保存するState構造体の定義、  
// =====  
struct TransformState  
{  
    → mtgb::EntityId parent;  
    → mtgb::Vector3 position;  
    → mtgb::Vector3 scale;  
    → mtgb::Quaternion rotate;  
};
```

生成されたコード

テンプレート内で使える変数(一部を抜粋)

class_name	クラス名
name_space	名前空間
metadata_type	クラスの属性 ※MT_COMPONENT(hoge)のMT_COMPONENT部分
meta_options	クラスの属性に設定されたオプション(配列) ※MT_COMPONENT(hoge)のhoge部分
parsed_file_relative_path	生成されたファイルから解析元のファイルへの相対パス

テンプレート内で使える変数(一部を抜粋)

properties	メンバの情報(配列)
- name	変数・関数名
- type_name	型名 * 名前空間を含まない。また、エイリアスでもそのまま
- name_space	名前空間
- metadata_type	メンバの属性 ※MT_PROPERTY(hoge)のMT_PROPERTY部分
- meta_options	クラスの属性に設定されたオプション(配列) ※MT_PROPERTY(hoge)のhoge部分

導入経緯

- このツールは、メインのゲーム制作を進める中で浮かび上がった課題を解決するために作成をした
- DirectXで3Dゲーム開発をするにあたって
以下のような機能を持つステージエディタを作成、使用していた
 - パラメータをエディタに表示、編集
 - エディタで加えた変更のUndo/Redo
 - JSONを用いたシリアルライズ、デシリアルライズ
- だが、クラスをエディタに登録する工程、設計に課題があった

課題

手動作業による開発効率の低下

Undo/Redoシステムの強引な差し込み、それによる複雑化

手動作業による開発効率の低下

- クラスにメンバを追加・変更するたび、エディタ表示用、シリアライズ処理用など、複数の箇所を同期させる必要がある。
- 手動コピー&ペーストによるタイムロスや、修正漏れによる不具合などが発生し、作業時間が削られていた。

一つメンバを追加すると...

```
int hoge;
```



```
struct TransformState  
{  
    mtgb::EntityId parent;  
    mtgb::Vector3 position;  
    mtgb::Vector3 scale;  
    mtgb::Quaternion rotate;  
};
```

3か所を編集

シリアライズ

デシリアライズ

エディタ表示

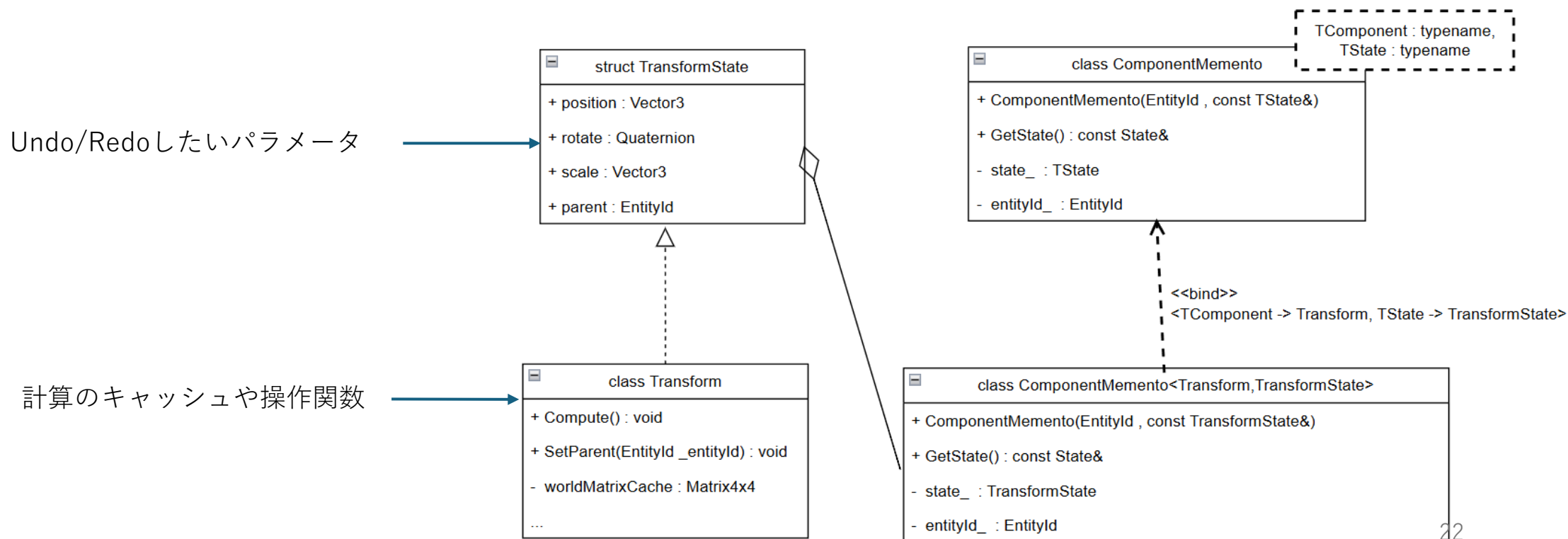
```
friend void to_json(nlohmann::json& _j, const Transform& _target)  
{  
    _j["parent"] = JsonSerializer::Serialize<mtgb::EntityId>(_target.parent);  
    _j["position"] = JsonSerializer::Serialize<mtgb::Vector3>(_target.position);  
    _j["scale"] = JsonSerializer::Serialize<mtgb::Vector3>(_target.scale);  
    _j["rotate"] = JsonSerializer::Serialize<mtgb::Quaternion>(_target.rotate);  
    _j[hoge] = ...  
}  
friend void from_json(const nlohmann::json& _j, Transform& _target)  
{  
    JsonSerializer::Deserialize<mtgb::EntityId>(_target.parent, _j, "parent");  
    JsonSerializer::Deserialize<mtgb::Vector3>(_target.position, _j, "position");  
    JsonSerializer::Deserialize<mtgb::Vector3>(_target.scale, _j, "scale");  
    JsonSerializer::Deserialize<mtgb::Quaternion>(_target.rotate, _j, "rotate");  
    _target.OnPostRestore();  
    _target.hoge = _j[hoge];  
}
```

```
RegisterShowFuncHolder::Set<Transform>([](Transform* _target, const char* _name)  
{  
    PropertyDisplayRegistry::Instance().ShowProperty(&_amp;_target->parent, "parent");  
    PropertyDisplayRegistry::Instance().ShowProperty(&_amp;_target->position, "position");  
    PropertyDisplayRegistry::Instance().ShowProperty(&_amp;_target->rotate, "rotate");  
    PropertyDisplayRegistry::Instance().ShowProperty(&_amp;_target->scale, "scale");  
    PropertyDisplayRegistry::Instance().ShowProperty(&_amp;_target->hoge, "hoge");  
});
```


Undo/Redoシステムの強引な差し込み、複雑化

以前の設計 (例:Transform)

状態を保存したいパラメータをState構造体に分離し、
Transformクラスがそれを継承し、ComponentMementoクラスがメンバ変数として保持している



設計の意図

クラスのパラメータと状態保存に同じ構造体を使うことで、修正箇所を減らすことができる

同じ構造体を使わない場合、一つメンバ変数を追加すると保存、復元処理、状態保存用の構造体を変更する必要がある

一つメンバを追加すると...

```
int hoge_;
```



```
class Transform
{
public:
    Vector3 position_;
    Vector3 scale_;
    // ...
}
```

メンバ追加

復元処理

保存処理

3か所を編集

```
struct TransformState
{
    Vector3 position;
    Vector3 scale;
    int hoge;
};

class Transform
{
public:
    using TransformMemento = ComponentMemento<Transform, TransformState>;
    void RestoreFromMemento(const TransformMemento& _memento)
    {
        const TransformState& state = _memento.GetState();
        position_ = state.position;
        scale_ = state.scale;
    } hoge_ = state.hoge;
    TransformMemento SaveToMemento()
    {
        // ...
        state.hoge = hoge_;
    }
};
```

設計の意図

継承ではなくコンポジションにすると...

- 既存のコードを大きく変更する必要がある
 - 修正箇所は数百に及ぶ
- アクセスが冗長になってしまう

```
class Transform
{
public:
    struct State
    {
        Vector3 position;
        Vector3 scale;
    };
private:
    State state;
};
```

State構造体をメンバとして持つ

継承

```
transform.position.x = 5.0f;
```



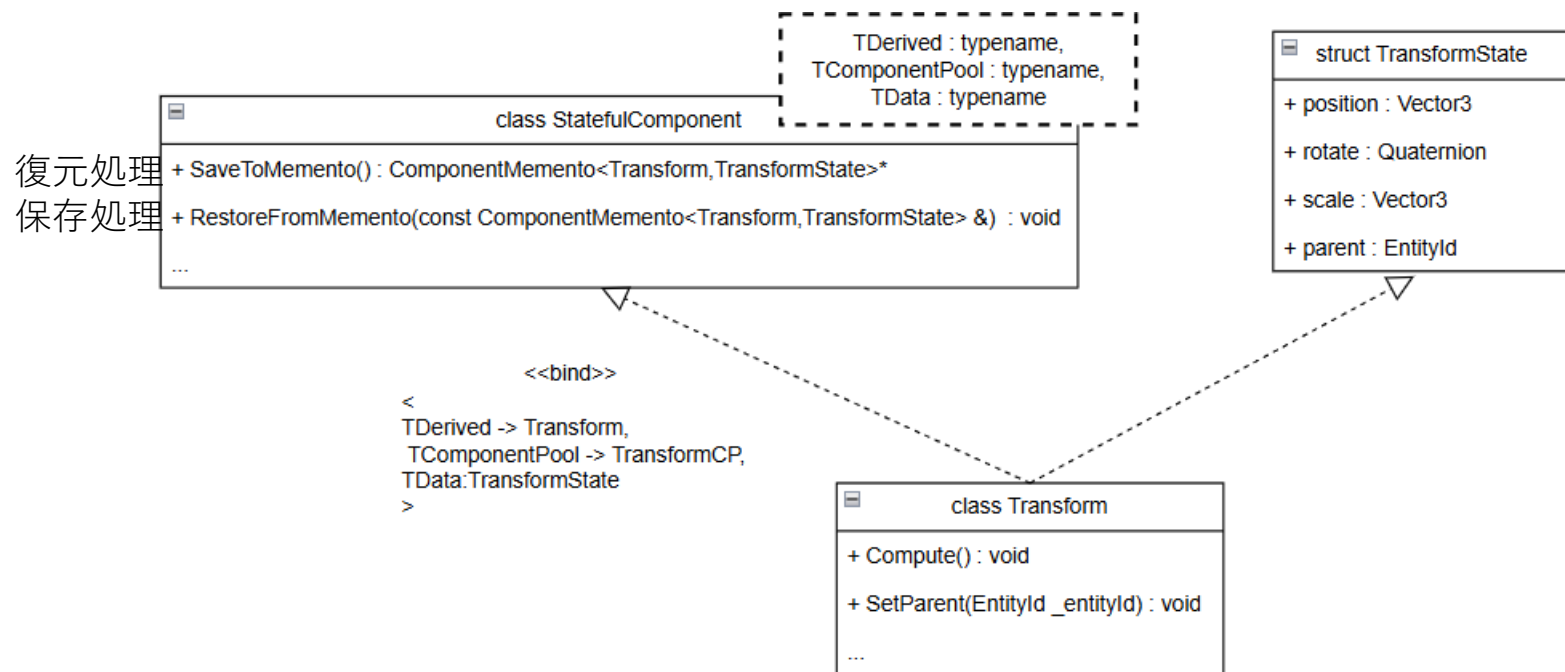
コンポジション

```
transform.state.position.x = 5.0f;
```

以前行っていたコマンドの保存・復元方法

以前の保存、復元方法

State構造体に加え、
StatefulComponent を継承することで
保存、復元処理が可能



以前の復元方法

RestoreFromMemento(復元処理)では、

StatefulComponent

↓ ①

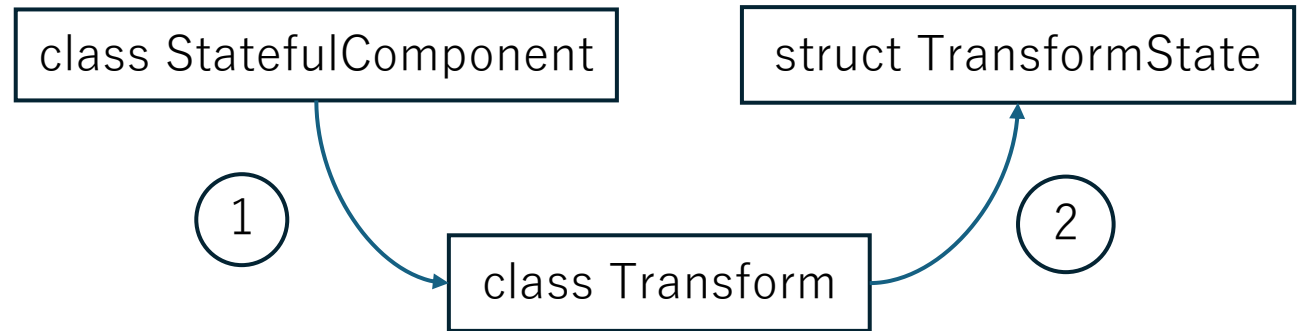
Transform(TDerived)

↓ ②

TransformState(TData)

とキャストして、メメントから復元をしている

```
void RestoreFromMemento(const Memento& _memento)
{
    ① auto& derivedRef = static_cast<TDerived*>(*this);
    const TData& data = _memento.GetData();
    ② static_cast<TData*>(derivedRef) = data;
    OnPostRestore();
}
```



問題点

- Undo・Redoのためだけに複雑、結合度の高い継承関係、クラステンプレートになっている
- ダウンキャストを行っているので、型安全性のためにconceptでの継承関係の保証をしたが
複雑なテンプレートが原因なのか、Intellisenseが誤検知を起こしてしまう
- そのためconceptを無効にしてしまい、型安全性が低い状態になっている

アプローチ

Undo/Redoシステムの不自然な差し込み、複雑化



State構造体、保存/復元処理を自動生成。Undo/Redoの差し込みを解消

```
class [[MT_COMPONENT()]] Transform :  
{  
public:  
    MT_GENERATED_BODY()  
    friend TransformCP;  
    using IComponent::IComponent;  
  
    [[MT_PROPERTY()]]  
    EntityId parent;  
    [[MT_PROPERTY()]]  
    Vector3 position;  
    [[MT_PROPERTY()]]  
    Vector3 scale;  
    [[MT_PROPERTY()]]  
    Quaternion rotate;  
};
```

Undo/Redo用の構造体

```
struct TransformState  
{  
    mtgb::EntityId parent;  
    mtgb::Vector3 position;  
    mtgb::Vector3 scale;  
    mtgb::Quaternion rotate;  
};
```

保存・復元処理

```
TransformMemento* SaveToMemento() .....  
{  
    OnPreSave(); .....  
    TransformState state; .....  
    state.parent = this->parent; .....  
    state.position = this->position; .....  
    state.scale = this->scale; .....  
    state.rotate = this->rotate; .....  
    return new Memento(GetEntityId(), state); .....  
}  
  
void RestoreFromMemento(const Memento& _memento) .....  
{  
    const TransformState& state = _memento.GetState(); .....  
    this->parent = state.parent; .....  
    this->position = state.position; .....  
    this->scale = state.scale; .....  
    this->rotate = state.rotate; .....  
    OnPostRestore(); .....  
}
```

アプローチ

手動作業による開発効率の低下



シリアライズ、エディタ表示処理を自動生成。ミス、作業時間を削減

```
class [[MT_COMPONENT()]] Transform :  
{  
    public:  
    → MT_GENERATED_BODY()  
  
    → friend TransformCP;  
    → using IComponent::IComponent;  
  
    → [[MT_PROPERTY()]]  
    → EntityId parent;  
    → [[MT_PROPERTY()]]  
    → Vector3 position;  
    → [[MT_PROPERTY()]]  
    → Vector3 scale;  
    → [[MT_PROPERTY()]]  
    → Quaternion rotate;  
};
```



```
friend void to_json(nlohmann::json& _j, const Transform& _target) {  
    → _j["parent"] = JsonConverter::Serialize<mtgb::EntityId>(_target.parent);  
    → _j["position"] = JsonConverter::Serialize<mtgb::Vector3>(_target.position);  
    → _j["scale"] = JsonConverter::Serialize<mtgb::Vector3>(_target.scale);  
    → _j["rotate"] = JsonConverter::Serialize<mtgb::Quaternion>(_target.rotate);  
}
```

```
RegisterShowFuncHolder::Set<Transform>({  
    → [(Transform* _target, const char* _name) {  
    → {  
    →     PropertyDisplayRegistry::Instance().ShowProperty(&_target->parent, "parent");  
    →     PropertyDisplayRegistry::Instance().ShowProperty(&_target->position, "position");  
    →     PropertyDisplayRegistry::Instance().ShowProperty(&_target->scale, "scale");  
    →     PropertyDisplayRegistry::Instance().ShowProperty(&_target->rotate, "rotate");  
    → }  
    → }  
});
```

成果

- 手書きによるタイムロスや書き漏らしによる不具合の排除
- 複雑、回りくどい設計の整理

に成功し、ゲーム本体のロジックに専念できる環境に一步近づいた。

苦勞した点

属性の解析

- 膨大な英語のドキュメントから属性を取得する方法を見つけられず、AST解析を使っているにも関わらず「正規表現」で強引に検索をしていた。
- 解析速度と汎用性に限界があり、`[[clang::annotate]]`をASTノードから取得する方法へ転換した。
- 公式ドキュメントを読み解く力、AIを活用して調査範囲のあたりをつけ、小さなコードで検証を繰り返すサイクルが重要だと学んだ。

工夫した点、アピールポイント

解析時間の緩和

- ファイル内容のハッシュ値を保存し、変更がないファイルは解析をスキップ
 - アルゴリズムをSHA256からxxHashに変えたことで、ファイル比較時間が1分から90ミリ秒に改善
- ファイル解析を並列実行。解析時間が約半分に減少
- 除外ディレクトリを設定して、フィルタリングできる

属性、メタ情報の付与

- MT_PROPERTYといった属性の付与に加え、なにかしらのメタ情報を付与できる
- メタ情報を付与することで、それに応じたソースコードの生成ができる

```
[[MT_PROPERTY(Hoge,Piyo)]]  
Vector3 position;
```

今後の課題、展望

生成内容の一新

現状

- シリアライズやエディタ表示のコードを「直接」生成している。
- 型情報やメタ情報を、ステージエディタ・ゲームエンジン側に渡さず、ツール側で自己完結している。

問題点

- メタ情報に応じた処理、例えば「エディタに編集不可で表示」「エディタに表示はせず、シリアライズのみ」などの細かい処理を生成するには、Scribanでは限界がある

対応策

- 処理ではなく型情報を生成して、ゲームエンジン側に渡すよう変更
 - 型名やメンバ変数のオフセット、サイズ、メタ情報など
- Undo/Redoやシリアライズは、型情報やサイズをもとに行うよう変更する